Qwen

Architecting Gentoo Security: A Sysadmin's Guide to SELinux Deployment, Policy Management, and Real-World Challenges

Foundational Principles: Understanding SELinux and Gentoo

To effectively deploy and manage Security-Enhanced Linux (SELinux) within the Gentoo ecosystem, one must first grasp its foundational principles and understand how they integrate with the core tenets of the Gentoo Linux distribution [18] [41]. SELinux is not an add-on security feature but a deep-seated capability of the Linux kernel itself, designed to provide a more robust and flexible framework for access control than what is offered by traditional permission models [6] [27]. Its design philosophy, centered on Mandatory Access Control (MAC), represents a significant departure from the standard security paradigm found in most operating systems [28] [30]. This section will demystify these concepts using simple analogies and then explore how Gentoo's emphasis on customization and control creates a powerful synergy with SELinux.

At its heart, SELinux is a framework built into the Linux kernel that allows for fine-grained access control beyond the standard file permissions [27]. To appreciate its value, it is crucial to differentiate between two primary models of access control: Discretionary Access Control (DAC) and Mandatory Access Control (MAC). DAC is the default model in Linux. In a DAC system, the owner of a resource—like a file or directory—has complete discretion over who can access it and what they can do with it [29]. Think of a file as a private house. The owner (user) decides who gets a key (read, write, execute permission) and can change those permissions at any time [33]. If you own a document, you can give your friend permission to read it, your colleague permission to edit it, and your sibling no permission at all. The "discretion" lies entirely with the owner. While flexible, this model has a critical weakness: it implicitly trusts the owner and the processes they run. If an attacker compromises a process running as a user—for example, through a vulnerability in a web browser—the malicious process inherits the user's discretionary permissions. It

could then read sensitive documents, modify system files, or exfiltrate data, all because the user had permission to do so [30].

SELinux introduces Mandatory Access Control to address this fundamental limitation. In a MAC system, access decisions are dictated by a centralized security policy defined by an administrator or system rules, rather than by the owner of the resource [31] [32]. Using our analogy, the house now has a strict set of rules enforced by a security guard (SELinux). The guard doesn't care who owns the house; he only cares about *what kind of person* (the process/subject) is asking to enter and *what kind of room* (the object/file). For example, a "delivery person" (`domain`) might be allowed to enter the "front door" (`type`) to leave a package (`{ write }`), but would be denied entry to the "owner's study" (`type`). Even if the owner says, "That delivery person can come in anytime," the guard enforces the rulebook. This policy is "mandatory" because it cannot be overridden by the resource owner or even by the superuser (root) in enforcing mode [27] [49]. SELinux operates on this principle, creating a much more robust security perimeter that is not dependent on the trustworthiness of individual users or their running processes [30].

The key components in an SELinux policy are subjects, objects, and domains. A subject is almost always a running process, such as a web server daemon, a database server, or a graphical application [31]. Every subject runs within a specific domain, which is essentially a confined execution environment defined by its SELinux context. An SELinux context is a label attached to every process and file, typically consisting of four parts: `user:role:type:range` [36] [52]. The most important part for policy enforcement is the type, which is conventionally suffixed with an underscore and a 't' (e.g., `httpd_t`, `sshd_t`, `firefox_t`) [39] [40]. An object is anything a subject might want to access, including files, directories, network ports, Unix sockets, and shared memory segments [31]. Like subjects, objects are also labeled with types (e.g., an executable file might have the type `httpd_exec_t`, and a user's home directory might have the type `user_home_dir_t`). An SELinux rule, therefore, looks something like: "Allow processes in the `domain` to perform `{ action }` on objects with the `object_type`." For example, a rule might state, "Allow processes in the `httpd_t` domain to read files labeled `httpd_log_t`." If a process tries to do something not explicitly permitted by the policy, SELinux generates a denial message, logs the event, and blocks the action [11]. This is the core mechanism of protection. The system operates in different modes: Enforcing, where denials are actually blocked; Permissive, where denials are logged but not blocked (useful for troubleshooting); and Disabled, where SELinux is completely turned off [48] [51].

This mandatory access control model aligns perfectly with the core tenets of Gentoo Linux. Gentoo is a versatile and fast, completely free Linux distribution geared towards

developers and network professionals [41] [42]. Unlike distributions that ship with a pre-configured, monolithic kernel, Gentoo empowers users to compile their own custom kernel from source code [14]. This gives users ultimate control over every aspect of the operating system, including hardware support and security features [42]. This philosophy of user-centric control makes Gentoo an ideal host for SELinux. Because Gentoo users build their kernels themselves, they can ensure that all necessary SELinux options are enabled without having to rely on a vendor patching a generic kernel [69]. The kernel configuration menu provides explicit options for SELinux, allowing the user to select "Y" (compile in) or "M" (compile as a module) [81]. Selecting "Y" for main SELinux options often automatically selects other required dependencies, simplifying the process and reducing the chance of missing a crucial piece [81]. This level of control ensures that the kernel itself is fully prepared to enforce SELinux policies from the moment it boots [2].

Furthermore, Gentoo's package management system, Portage, plays a vital role in SELinux integration [18]. When a user installs a software package using the `emerge` command, Portage is responsible for more than just copying files; it executes scripts and logic to set up the system correctly [59]. This includes applying the correct SELinux contexts to files as part of the installation process. Many popular applications available in Gentoo's vast Portage tree, such as Apache or NGINX, come with SELinux profiles [11] [23]. These profiles contain the necessary information for Portage to know exactly how to configure the application securely within the SELinux framework. For example, when emerging `www-client/firefox`, Portage would be responsible for labeling the Firefox binary with the `firefox_exec_t` type, placing profile directories in locations with the `firefox_home_t` type, and ensuring configuration files get the appropriate `etc_t` or similar type [22]. The existence of SELinux-aware applications implies that Portage has mechanisms, likely tied to USE flags or internal ebuild metadata, to handle these special security requirements during installation [42]. Enabling a `selinux` USE flag for a package instructs Portage to perform a more secure installation tailored to SELinux's confinement model, potentially by applying patches or modifying the installation paths to comply with policy requirements [42].

Finally, the interaction between SELinux and systemd, the service manager used by default in most modern Linux systems, is seamless yet requires understanding [41]. Systemd is responsible for managing the lifecycle of services—how they are started, stopped, restarted, and monitored [75]. SELinux, on the other hand, defines the boundaries of what a service is allowed to do once it is running [31]. A common point of failure occurs when a service fails to start. The root cause may not be a syntax error in the systemd unit file but rather a SELinux denial that prevents the service from

performing a necessary initialization action, such as binding to a network port, reading its configuration file, or creating a lock file [22]. Diagnosing such issues involves a dual-check process: first, inspecting the output of `systemctl status servicename.service` to see if systemd reports a startup failure, and second, checking the audit logs for AVC denials related to the service's startup sequence [11]. The combination of systemd's declarative service management and SELinux's mandatory policy enforcement creates a powerful defense-in-depth strategy for securing system services. The administrator defines *what* the service should do (via the systemd unit) and *where* it is allowed to operate (via the SELinux policy), resulting in a system that is both functional and resilient against unauthorized access.

Pre-Installation Planning and Environment Validation

Before writing a single command to install Gentoo with SELinux, several critical architectural decisions must be made. These choices affect the entire installation and cannot be easily changed later. Making informed decisions at this stage prevents future conflicts, especially when integrating complex technologies like ZFS, UEFI bootloaders, and proprietary drivers. This planning phase is the foundation upon which a stable and secure SELinux-enforced system is built. The provided sources highlight several non-obvious but crucial setup considerations that differentiate a successful Gentoo+SELinux installation from a problematic one.

One of the most frequently overlooked decisions is the location of the EFI System Partition (ESP). The recommendation is to mount the ESP at `/boot/efi` rather than the traditional `/boot`. This separation is particularly important for systems using ZFSBootMenu, the bootloader of choice for many ZFS users. Mounting the ESP at `/boot` can create namespace conflicts during early boot, as ZFS utilities may attempt to manipulate the root filesystem, leading to unpredictable behavior and ZFSBootMenu failures [15]. By mounting the ESP at `/boot/eff`, a clean separation is maintained between the UEFI firmware's expectations and the OS' boot management responsibilities. This ensures that ZFSBootMenu can discover datasets cleanly and that `systemd` and `installkernel` operate without polluting ZFS-managed datasets [63]. The validation script provided in the initial research underscores the importance of this decision by including a check for the ESP mount point as part of the overall environment verification process [63].

Another critical decision concerns the integration of ZFS and SELinux. Specifically, the choice between building ZFS directly into the kernel (`CONFIG_ZFS=y`) versus using the Dynamic Kernel Module Support (DKMS) pathway via the `sys-fs/zfs` package is paramount. The DKMS approach is strongly recommended for SELinux environments. Using the built-in kernel ZFS path complicates the initramfs creation process, requiring manual intervention with `dracut` and often resulting in fragile compatibility with ZFSBootMenu [16]. Conversely, the DKMS pathway, managed by `sys-fs/zfs-dracut`, integrates seamlessly with the Gentoo build system, ensuring that the initramfs contains the necessary ZFS modules automatically [76]. This method is cleaner, more reliable, and better supported by the SELinux policy for DKMS modules [69]. Furthermore, it avoids potential boot-time complexities and ensures the security policy is available from the earliest moment the system starts [2]. The plan should therefore explicitly call for commenting out or removing the `kernel-builtin-zfs` USE flag for the `sys-fs/zfs` package to force the use of the DKMS pathway [42].

The selection of the appropriate SELinux policy package is the next step in pre-installation planning. Gentoo offers several policy packages, each with a different scope. The primary options include `app-admin/selinux-base`, which provides the core framework; `sec-policy/selinux-targeted`, which confines key network services and daemons; and `sec-policy/selinux-desktop`, which adds profiles for common desktop applications like web browsers and media players [11] [23]. For a general-purpose desktop or server, starting with a combination of the targeted and desktop policies is a logical approach, balancing security with usability. The installation guide consolidates this by recommending the emergence of all three packages: `sec-policy/selinux-base`, `sec-policy/selinux-targeted`, and `sec-policy/selinux-desktop` [11]. This ensures that the system is equipped with a comprehensive baseline policy covering both server and desktop use cases from the outset.

Finally, the environment itself must be validated before proceeding with the installation. This involves confirming internet connectivity for fetching packages, verifying the presence of target storage devices (like NVMe drives), and ensuring that the user understands the destructive nature of the installation process. The provided installation script demonstrates a practical approach to this validation, using commands like `ping` to check network access, `lsblk` to confirm drive detection, and a simple `read` prompt to require explicit confirmation before wiping target disks [63]. This proactive validation prevents wasted effort and ensures that the live environment is ready for the demanding tasks of partitioning, ZFS pool creation, and chrooting into the new system.

Decision Area	Recommended Choice	Rationale
ESP Mount Point	`/boot/efi`	Prevents namespace conflicts with ZFS utilities and ensures ZFSBootMenu compatibility [15] [63].
ZFS Module Pathway	`sys-fs/zfs` (DKMS)	Provides automatic initramfs integration via `zfs-dracut`, ensuring reliability and compatibility [16] [76].
SELinux Policy Base	`sec-policy/selinux-targeted`	Confining key services provides a strong security posture for most use cases [11] [23].
SELinux Desktop Support	`sec-policy/selinux-desktop`	Adds profiles for GUI applications, improving usability on desktop systems [11] [23].

By addressing these architectural questions and validating the environment beforehand, the system administrator sets the stage for a smooth installation. These preparatory steps, though seemingly minor, are critical for avoiding the common pitfalls that plague SELinux installations on complex Gentoo systems. They ensure that the underlying infrastructure—bootloader, filesystem, and core security policy—is correctly configured to support the high-security goals of the project.

Kernel Configuration for SELinux: Deep Dive

On Gentoo, you compile your own kernel. This is a powerful advantage for SELinux because you can enable exactly the options you need—no more, no less. A misconfigured kernel can lead to catastrophic failures, such as SELinux failing to load at boot, critical denials due to missing hooks, or boot failures with ZFS or NVIDIA modules. This section provides a deep-dive into the critical kernel configuration options, explaining their purpose and offering recommendations tailored to a Gentoo system with ZFS, SELinux, and NVIDIA drivers. The goal is to create a kernel that is both secure and fully functional, with SELinux active from the earliest possible moment.

The process begins after emerging the desired kernel sources, such as `cachyos-sources`, and selecting it with `eselect kernel` [18]. Navigate to the kernel source directory (`/usr/src/linux`) and initiate the configuration menu with `make menuconfig` [81]. It is advisable to start with an existing `.config` file as a base and update it with `make LLVM=1 olddefconfig`, which intelligently handles changes between kernel versions [81]. Alternatively, a fresh configuration can be started with `make LLVM=1 menuconfig`, though this requires manually enabling all necessary options [81].

The most critical section is located under `General setup` -> `Security options`. Here, the following options must be meticulously configured:

```
[*] Enable different security models
 [*] SELinux Support
  [*] Enable runtime disabling of SELinux
  [ ] Enable secure boot signing (optional)
```

The `CONFIG_SECURITY_SELINUX` option is the cornerstone of the entire system. It is highly recommended to select "Y" (built-in) rather than "M" (module) [81]. Compiling SELinux directly into the kernel ensures that the security policy is loaded and active from the very beginning of the boot process, eliminating any window of opportunity where the system is running without mandatory access controls [2]. The `CONFIG_SECURITY_SELINUX_DISABLE` option should also be enabled ("Y"). This allows the administrator to disable SELinux at runtime using the `setenforce` command, which is invaluable for troubleshooting and recovery [65] [73].

Beyond the main SELinux switch, several other options must be considered. `CONFIG_SECURITY_SELINUX_BOOTPARAM` is often selected automatically when `SELinux Support` is enabled and allows for passing `selinux=0` as a kernel parameter, providing a fallback for boot failures [81]. Options like `CONFIG_SECURITY_SELINUX_DEVELOP` should be left disabled ("N") for production systems, as they add debugging features that are not needed in a stable environment [81].

For systems using ZFS with native encryption, specific cryptographic algorithms must be enabled. Navigate to `Cryptographic API` and ensure the following are selected:

```
<*> XTS support
<*> AES cipher algorithms
<*> SHA384 and SHA512 digest algorithm
<*> CRC32c CRC algorithm
```

These are not optional. `XTS` is the cipher mode used by ZFS encryption, `AES` is the encryption algorithm itself, `SHA512` is used for integrity verification, and `CRC32c` is for checksumming data [37] [38]. Without them, encrypted ZFS datasets will fail to function.

UEFI boot parameters are also critical for ZFSBootMenu compatibility. Under `Processor type and features`, enable:

```
[*] EFI runtime service support
 [*] EFI stub support
```

The EFI stub allows the kernel image to be loaded directly by the UEFI firmware, which is precisely how ZFSBootMenu chainsloads the kernel from the FAT32 ESP [69]. This is essential for a secure and predictable boot process.

Finally, graphics driver options must be handled carefully. Under `Device Drivers -> Graphics support`, the following settings are relevant:

```
<*> Direct Rendering Manager (DRM)
<*> NVIDIA DRM
 [*] NVIDIA DRM modesetting support
```

It is critically important not to enable the kernel's built-in NVIDIA open kernel modules. The Gentoo way is to use the `nvidia-drivers` package with the `open-kernel-modules` USE flag, which manages the NVIDIA modules via DKMS [69]. Enabling the kernel's version would create a conflict, as two different mechanisms would try to load the same driver. The `nvidia-drm` module, compiled as a separate module, works correctly alongside the DKMS-managed userspace driver and is necessary for proper display management [69].

After configuring the kernel, the compilation process is straightforward: `make LLVM=1 -j$(nproc)` followed by `make LLVM=1 modules_install` and `make LLVM=1 install` [81]. The `LLVM=1` flag speeds up compilation on modern Gentoo systems. After installation, the bootloader (GRUB) must be updated to point to the new kernel and initramfs images. The final step is to verify the configuration by checking the saved `.config` file and using `zgrep` to confirm that `CONFIG_SECURITY_SELINUX=m` is set, indicating the feature is enabled [81]. This meticulous kernel configuration is the bedrock upon which a secure and stable Gentoo+SELinux system is built.

Initial System Setup and Filesystem Relabeling

With the kernel compiled and the basic environment prepared, the initial system setup begins. This phase transforms a bare-bones Gentoo stage3 archive into a fully functional, SELinux-enforced system. The process involves configuring Portage for SELinux integration, installing the necessary policy packages, activating SELinux at boot time, and, most importantly, performing the lengthy filesystem relabeling process. Each step is critical and must be executed in order to establish a consistent and secure baseline for the new system.

The first task within the chroot environment is to configure Portage to build SELinux-aware packages. This is achieved primarily through the use of USE flags. The global `selinux` USE flag should be added to `/etc/portage/make.conf` to enable SELinux support in packages by default [42]. Additionally, specific USE flags for key components like `kernel-open` for NVIDIA drivers and `cuda` should be added to ensure they are built with the necessary SELinux compatibility patches [69]. Package-specific USE flags can be managed in `/etc/portage/package.use/` to fine-tune the build for individual applications [42]. Keywording for testing packages is also necessary for some SELinux-related tools, which can be managed in `/etc/portage/package.accept_keywords/` to ensure access to the latest versions and security fixes [69].

Next, the core SELinux policy packages must be emerged. The installation guide recommends a suite of packages that provide a comprehensive security framework:

```
emerge --ask app-admin/selinux-base \
               sec-policy/selinux-base \
               sec-policy/selinux-targeted \
               sec-policy/selinux-desktop
```

`app-admin/selinux-base` provides the fundamental libraries and tools [11]. `sec-policy/selinux-targeted` contains the policy for confining network services, while `sec-policy/selinux-desktop` adds rules for common desktop applications like Firefox and SDDM [11] [23]. Emerging these packages populates `/etc/selinux/` with the policy configuration files and loads the initial policy into the kernel.

Once the packages are installed, SELinux must be enabled to run in enforcing mode at boot. This is typically accomplished by a helper script provided by the SELinux packages, often named `selinux-enable` [63]. Running this script as root modifies the bootloader's configuration to pass the necessary kernel parameters, such as `security=selinux` and `selinux=1`, ensuring that SELinux is active upon every subsequent reboot [63]. The script may also edit `/etc/selinux/config` to persistently set `SELINUX=enforcing` [63]. After running the enable script, a reboot is required for the changes to take effect.

Upon rebooting into the newly configured SELinux-enabled kernel, the system enters its most time-consuming initialization step: filesystem relabeling. At this point, SELinux is active and enforcing, but the vast majority of files on the system still lack proper SELinux context labels [77]. Without these labels, SELinux cannot determine what a process is allowed to do with a file, rendering the system largely insecure and unstable as countless

denials occur [64]. Therefore, the system must go through a comprehensive process to assign the correct type to every single file based on the installed policy. This is achieved by triggering the relabel process.

The standard way to trigger a full relabel is to create a special marker file called `/.autorelabel` in the root directory of the root filesystem [63]. This file signals to the boot scripts that a relabel is needed. The command is simply `touch /.autorelabel` [63]. After creating this file, a final reboot is required. The system will then boot, immediately switch to permissive mode to avoid blocking operations during relabeling, and run a relabeling utility (often `fixfiles`) that walks through every directory and file on the mounted filesystems, assigning the correct context according to the rules in `/etc/selinux/targeted/contexts/files/file_contexts` [63]. Administrators should be warned that this operation can take a very long time, especially on systems with large amounts of data, as it involves walking through every directory and file on the specified partition [64]. During this process, the system logs the context assignments being made. It is absolutely critical that this step completes fully before attempting to use the graphical desktop or critical services, as premature use will result in numerous denials and application failures. Interrupting the process mid-way can leave the filesystem in an inconsistent state, potentially causing unpredictable behavior. Once the relabeling is complete, the system will reboot automatically, and SELinux will be in enforcing mode with a fully labeled filesystem, marking the successful completion of the initial setup.

Step	Command / Action	Description
1. Kernel Config	`make menuconfig`	Enable "Security options" -> "Enable SELinux Support" and select "Y". [81]
2. Compile Kernel	`make LLVM=1 && make modules_install && make install`	Build and install the new kernel image and modules. [81]
3. Install SELinux Packages	`emerge --ask sec-policy/selinux-base sec-policy/selinux-targeted sec-policy/selinux-desktop`	Install the SELinux policy and management tools via Portage. [11] [23]
4. Enable at Boot	`selinux-enable`	Configure the bootloader to start the system in enforcing mode. [63]
5. Reboot for SELinux Activation	`reboot`	Start the system with the new kernel and SELinux enabled. [63]
6. Trigger Relabel	`touch /.autorelabel`	Create the autorelabel marker file to initiate filesystem labeling. [63]
7. Final Reboot	`reboot`	Complete the relabeling process and enter the final enforcing state. [63]
8. Verify Status	`getenforce`	Confirm SELinux is running in "Enforcing" mode. [81]

Daily Administrative Workflows and Policy Management

Mastering SELinux on a Gentoo system transcends the initial setup; it requires a deep understanding of ongoing policy management and the daily administrative workflows that keep a confined environment both secure and functional. The power of SELinux lies not just in its ability to block unauthorized access, but in the administrator's ability to teach it what *is* authorized. This is achieved through a tiered approach that prioritizes simplicity and safety. The primary tools for this task are `semanage` for managing static policy attributes like file contexts and booleans, and `audit2allow` for generating dynamic, custom policy modules when necessary. A disciplined administrator follows a clear hierarchy to resolve denials, minimizing risk and maintaining a lean security posture.

The first line of defense against most denials is the use of SELinux booleans. Booleans are named switches that allow an administrator to toggle entire categories of access on or off without modifying the underlying policy [19]. This is the lowest-risk, most manageable method for resolving common permission issues. For example, if a web browser needs to print, there may be a boolean like `allow_httpd_can_sendmail` that, when enabled, grants the necessary permission instantly [19]. The workflow is simple: check for an applicable boolean using `getsebool -a`, filter the output for the relevant service name, and if one is found, enable it permanently with `setsebool -P <boolean_name> on` [20]. The `-P` flag is critical, as it makes the change persistent across reboots [20].

If a denial is related to accessing a file in a new or non-standard directory, the next step is to manage file contexts with `semanage`. By default, files placed in `/srv/www/myapp` will not have the `httpd_sys_content_t` type required by the web server. Instead of resorting to broad allowances, `semanage fcontext` should be used to teach SELinux about this new location. The command `semanage fcontext -a -t httpd_sys_content_t "/srv/www/myapp(/.*)?"` tells the relabeling process to apply the `httpd_sys_content_t` type to the directory and all its contents [70]. After adding the rule, `restorecon -Rv /srv/www/myapp` must be run to apply the new context to the existing files [78]. This is the preferred solution for managing application data and web content stored outside standard paths.

Only after exhausting the above two options—checking for booleans and creating file context rules—should an administrator resort to `audit2allow`. This tool should be viewed as a last resort for truly exceptional cases where no other solution exists [44]. The

workflow begins by reproducing the denial and capturing the relevant message from the audit log, typically `/var/log/audit/audit.log` [11]. The command `ausearch -m avc -ts recent` is effective for this purpose [11]. This output is then piped to `audit2allow` with the `-M` flag to generate a new policy module: `audit2allow -M myapp_fix` [23]. This creates two files: `myapp_fix.te` (the human-readable Type Enforcement source code) and `myapp_fix.pp` (the compiled policy module) [23]. Before installing, it is imperative to manually review the `.te` file to ensure the generated rule is as minimal and specific as possible [44]. The final step is to install the module with `semodule -i myapp_fix.pp` [23].

This systematic approach—using booleans for toggles, `semanage` for contexts, and `audit2allow` for custom rules—is the cornerstone of effective SELinux administration. It ensures that the minimum necessary privileges are granted, adhering to the principle of least privilege. The following table summarizes the primary commands and their functions.

Category	Command	Purpose	Example Usage
Status	`sestatus`	Display detailed SELinux status and configuration.	`sestatus`
	`getenforce`	Check the current SELinux enforcing mode.	`getenforce`
	`setenforce`	Temporarily switch SELinux mode (1=enforcing, 0=permissive).	`setenforce 0`
Denial Analysis	`ausearch`	Search the audit log for specific events (e.g., AVC denials).	`ausearch -m avc -ts recent`
	`journalctl`	View system logs, including audit messages.	`journalctl -t audit`
Context Inspection	`ls -Z`	Show the SELinux context of a file or directory.	`ls -Z /usr/bin/firefox`
	`ps -eZ`	Show the SELinux context of all running processes.	`ps -eZ | grep sshd`
Policy Management	`semanage`	Manage policy details without editing files directly.	`semanage boolean -l`
	`setsebool`	Change the state of a SELinux boolean.	`setsebool -P httpd_can_network_connect_db 1`
	`audit2allow`	Generate policy from audit log denials.	`audit2allow -M myapp_fix < /tmp/denial.log`
	`semodule`	Manage loaded SELinux policy modules.	`semodule -l | grep myapp_fix`
Context Restoration	`restorecon`	Restore SELinux contexts on files based on policy.	`restorecon -Rv /home/ahsan`

By mastering this toolkit, a Gentoo administrator can effectively manage a SELinux-enforced system, quickly resolving common access issues, precisely tailoring policies for custom applications, and maintaining the integrity of file contexts throughout the system's lifecycle.

Practical Scenarios: Desktop and Server Use Cases

While the foundational mechanics of SELinux are universal, its practical application takes on distinct characteristics depending on whether the Gentoo system is configured as a desktop workstation or a server. For a system administrator, understanding these different use cases is key to leveraging SELinux for tangible security benefits in real-world scenarios. Whether it's ensuring a web browser prints a document securely or confining a web server to serve only its designated content, SELinux provides a powerful means of enforcing security policies. The iterative workflow of observing, analyzing, and teaching SELinux remains the essence of management, transforming potential frustration into a robust security posture.

On a **desktop** system, the primary challenge is balancing stringent security with the usability that users expect. Applications, especially graphical ones like web browsers, media players, and office suites, must "Just Work" out of the box. Fortunately, many popular applications available in Gentoo's Portage tree come with SELinux support, significantly reducing initial friction [11]. Emerging a package like `www-client/firefox` with the `selinux` USE flag instructs Portage to apply the correct SELinux contexts during installation, ensuring that upon first launch, Firefox is already running in its designated `firefox_t` domain with a reasonable set of permissions for browsing, managing its profile, and communicating over the network [22].

Despite this good initial state, issues will inevitably arise. A common desktop denial involves a web browser trying to print a document. Printing often requires communication with a CUPS daemon, which runs in its own SELinux domain (`cupsd_t`). The denial might appear as the `firefox_t` process being denied a `name_connect` permission to a CUPS socket. The first step to resolve this is to check for an appropriate boolean. There may be a boolean like `allow_firefox_can_print` that, if enabled, would grant this permission. If no such boolean exists, the administrator can use `audit2allow` to generate a custom policy allowing the `firefox_t` domain to connect to the CUPS runtime socket [22]. Another frequent scenario on a desktop is a web browser or email client trying to access files in the user's home directory outside of its

designated profile directory. By default, domains like `firefox_t` are confined from accessing most user files, a security feature designed to prevent a compromised browser from snooping on personal documents [22]. If an application legitimately needs to open a file from the "Downloads" folder, the administrator can use `semanage` to create a persistent rule allowing the `firefox_t` domain to read files within the `user_home_dir_t` type. For example, `semanage fcontext -a -t user_home_dir_t "/home/username/Downloads(/.*)?"` followed by `restorecon -R /home/username/Downloads` [22]. This provides a targeted allowance that respects the overall security posture of the system while solving the immediate usability problem.

For **server** use cases, the goal is to confine services to the absolute minimum set of privileges necessary to perform their function, a principle known as least privilege. This is where SELinux excels at containing potential breaches. A common example is setting up a web server like Apache (`httpd`) or Nginx. When emerging the web server package from Portage, enabling the `selinux` USE flag ensures it is integrated into the system's security policy [69]. By default, the `httpd` domains are configured to serve content from `/srv/www/`, bind to standard HTTP (port 80) and HTTPS (port 443) ports, and have very limited access to the rest of the filesystem. If an administrator wants to serve content from a non-standard location, such as `/var/www/example.com`, they would first use `semanage fcontext` to tell SELinux to label that directory and its contents with the `httpd_content_t` type [69]. The command would look something like `semanage fcontext -a -t httpd_content_t "/var/www/example.com(/.*)?"`. After adding the rule, `restorecon -R /var/www/example.com` must be run to apply the new contexts to the existing files [70]. If the web server needs to perform dynamic functions, such as executing CGI scripts or connecting to a backend database, additional SELinux booleans may be required. For instance, the `httpd_can_network_connect_db` boolean might need to be enabled to allow the web server to communicate with a database service running on another machine [20].

Another critical server component is the Secure Shell (SSH) daemon (`sshd`). The `sshd_t` domain is designed to be highly restrictive. It can listen on port 22, read its host keys and configuration files, and create sessions for authenticated users. However, by default, it is prevented from performing actions like mounting filesystems or accessing raw network devices [22]. If a system administrator needs to allow SSH access to a specific service or requires a user to have extended capabilities via SSH (e.g., for SFTP-only access to a project directory), they must carefully evaluate the risk and use `audit2allow` to generate the minimal policy necessary to grant that access. For example, to allow a user's shell to access a specific project directory outside their home

folder, a policy could be written to permit the `sshd_t` domain to `search` the parent directory and `read_dir_it` the project directory [22]. This targeted approach confines the extended capability to only what is strictly necessary, minimizing the potential attack surface. Regardless of the use case, the iterative workflow of observing, analyzing, and teaching SELinux remains the essence of management. When an application fails to work as expected, the administrator follows a logical path: check the audit logs for denials, diagnose the denial message to understand the source domain, target type, and requested action, and then choose the appropriate remediation strategy. This process not only fixes the immediate problem but also contributes to a more robust and secure system over time. Gentoo's flexibility shines here, as the user can emerge packages with SELinux support, fine-tune contexts with `semanage`, and build custom policies with `audit2allow` to match their specific security requirements, whether for a personal laptop or a hardened server. This systematic approach transforms SELinux from a source of frustration into a powerful ally in the pursuit of system security.

Advanced Integration Scenarios and Troubleshooting

While SELinux provides formidable security enhancements, its implementation in a highly customizable environment like Gentoo is not without challenges. Certain software, hardware configurations, and advanced technologies require special attention and a deeper level of expertise to integrate successfully. System administrators must be aware of these complexities and possess a systematic approach to troubleshooting to maintain a stable and secure system. This section delves into some of the most notorious areas of difficulty, including the notorious integration with proprietary graphics drivers like NVIDIA, the nuances of managing SELinux in containerized environments, and strategies for maintaining long-term system stability. These topics represent the frontier of SELinux administration and are critical for any administrator looking to deploy SELinux in a production Gentoo environment.

One of the most challenging areas for SELinux is the integration with proprietary hardware drivers, most notably the NVIDIA graphics driver. These drivers are closed-source kernel modules that perform low-level hardware manipulation, which often conflicts with SELinux policies that restrict memory access, device file operations, and module loading [69]. On many distributions, getting NVIDIA drivers to work with SELinux requires applying specific patches or modifications to the SELinux policy. For Gentoo users, this means that simply emerging the driver package via Portage may not be sufficient [69]. The most reliable approach is to consult Gentoo's extensive documentation,

forums, and Bugzilla for solutions specific to the version of the driver and the kernel in use [69]. It is common to find ebuilds (Gentoo's package descriptions) that include patches or specific USE flags designed to accommodate SELinux. For example, there might be an `nvidia-drivers` USE flag like `selinux-support` that applies the necessary policy tweaks during installation or compilation [69]. If no official solution exists, the administrator must analyze the denial messages generated when the driver loads. These denials, typically found in `/var/log/audit/audit.log`, will indicate what specific operation the `nvidia_modprobe_t` or `nvidia_uvm_t` domain is being blocked from performing (e.g., accessing `/dev/nvidiactl`, mapping memory) [69]. The administrator can then use `audit2allow` cautiously on these denials to create a custom policy allowing the required operations. A pragmatic, real-world troubleshooting tactic is to temporarily set the offending domain to permissive mode using `semanage permissive -a nvidia_uvm_t`. This restores functionality while a more permanent solution is sought, allowing the system to remain usable while further investigation is conducted [69].

Another advanced topic is the use of containers, such as Docker or Podman. Podman, in particular, has gained popularity as a daemon-less container engine that is designed to be more compatible with the Linux ecosystem [22]. Running containers introduces another layer of complexity to SELinux management. The host system's SELinux policy must first allow the container engine daemon (which might run in a `container_t` domain) to perform its duties, such as creating namespaces, mounting volumes, and managing network interfaces [69]. Furthermore, the administrator must decide how to label the containers themselves. Should they run in a confined domain like `container_t` or a more restrictive one like `svirt_lxc_net_t` for LXC-based containers? The choice depends on the desired level of isolation. Volume mounts are a frequent source of problems. When a host directory is mounted into a container, SELinux contexts can cause "permission denied" errors because the container's domain may not have access to the host's file type. The solution often involves using the `z` or `Z` options with Docker/Podman volume mounts, which tells the container runtime to automatically relabel the mounted content with the appropriate type for the container domain [69]. Alternatively, the `semanage` tool can be used to create custom file context rules for shared data directories on the host. Networking between containers and between containers and the host is also governed by SELinux network policies, with booleans like `container_can_connect` often playing a key role [69]. Gentoo's flexibility allows for compiling the kernel and SELinux policy with container-specific options, but it places the responsibility of correct configuration squarely on the user.

Beyond specific hardware and virtualization technologies, long-term maintenance of a SELinux-enabled Gentoo system involves several key practices to ensure continued stability and security. First, as the system evolves and new packages are emerged, it is important to regularly monitor for new denials. A new application might introduce behaviors that were not anticipated by the existing policy, and catching these issues early prevents them from becoming larger problems. Regularly reviewing the output of `sestatus` or searching the audit logs can help identify these regressions. Second, when upgrading the base SELinux policy package (e.g., `sec-policy/selinux-targeted`), it is possible that the new policy will introduce stricter rules or remove old allowances, which could break existing functionality. In such cases, the administrator will need to use the `audit2allow` workflow to create new custom policies to restore the desired functionality, just as they did during the initial setup [69]. Third, because Gentoo users compile their own kernels, updating the kernel is a regular activity. While the SELinux options are generally stable, a major kernel upgrade could theoretically introduce changes that affect policy behavior. After a kernel update, it is prudent to perform a full relabel (`fixfiles relabel /`) to ensure all file contexts are consistent with the new kernel's expectations and to catch any inconsistencies that may have arisen [15]. Finally, performance implications are a concern for any security technology. While the provided sources do not contain quantitative data on CPU or memory overhead, it is generally considered negligible on modern hardware for most workloads [69]. Any measurable impact would be workload-dependent and would need to be benchmarked on a case-by-case basis. For high-load scenarios, administrators should be aware that every system call is subject to an extra check by the SELinux kernel module. However, for the vast majority of desktop and server applications, the performance cost is far outweighed by the security benefits. The key is to maintain a well-tuned policy that is as minimal as possible, avoiding overly broad rules that could lead to unnecessary denials and retries. By combining proactive monitoring, careful policy management, and a systematic troubleshooting methodology, a Gentoo administrator can successfully navigate these advanced scenarios and maintain a system that is both highly secure and reliably functional.

Long-Term Maintenance and Strategic Overview

The journey of deploying SELinux on a Gentoo system does not end after the initial setup and relabeling. Effective long-term maintenance is what separates a system that is merely functional from one that is genuinely secure and resilient. This involves a continuous cycle of monitoring for new denials, carefully managing policy upgrades, and adapting to

changes in the system, such as kernel updates. A strategic overview of the Gentoo approach, compared to other distributions, highlights the unique trade-offs involved, ultimately positioning Gentoo as the premier choice for administrators who prioritize ultimate control and customization over convenience.

A cornerstone of long-term maintenance is routine auditing. Denials are not just errors; they are valuable feedback from the security policy. Administrators should establish a regular schedule, such as weekly or monthly, to review the output of `sestatus` and search the audit logs for new or recurring denials [11]. Tools like `ausearch -m avc -ts week` can isolate recent violations, and piping them to `audit2why` can provide human-readable explanations of why an access was denied [44]. This proactive monitoring helps catch issues early, before they escalate into critical failures. When new denials appear after emerging a new package, they should be investigated using the familiar workflow: check for a relevant boolean, verify file contexts, and only then generate a custom policy with `audit2allow` if absolutely necessary [44]. Over time, a sysadmin will develop a mental library of common denials and their fixes, but for new or uncommon applications, the `audit2allow` workflow remains the indispensable tool for success [11].

Policy upgrades are another critical maintenance task. As security vulnerabilities are discovered upstream, the SELinux policy packages in Portage are updated. These updates can sometimes introduce stricter rules or remove allowances that were present in older versions of the policy. When such an upgrade breaks existing functionality, the administrator must use the `audit2allow` workflow again to create a new custom policy that teaches the updated policy the specific access it now needs [69]. This reinforces the importance of documenting custom policies. Saving the `.te` source files for all custom modules provides a clear record of the rationale behind each rule and makes it easier to adapt them to future policy changes [44]. Periodic audits of custom policies and modified booleans using scripts can help ensure the system's configuration remains transparent and intentional [70].

Kernel updates are a regular part of life on Gentoo. While SELinux policy behavior is generally stable across kernel versions, a major update could theoretically introduce changes that affect policy behavior. After a kernel update, it is prudent to perform a full relabel (`fixfiles relabel /`) to ensure all file contexts are consistent with the new kernel's expectations and to catch any inconsistencies that may have arisen [15]. This practice, combined with regular monitoring, forms a robust strategy for maintaining system integrity over time.

Ultimately, the choice of a distribution for a SELinux-enforced system comes down to the administrator's priorities. The Gentoo approach, characterized by manual configuration, deep integration, and granular control, offers profound benefits for the knowledgeable administrator. The ability to compile a kernel with only the SELinux options needed, to emerge packages with SELinux features enabled via USE flags, and to build a system with a policy precisely tailored to its intended purpose results in a highly optimized and secure environment [18] [42]. This contrasts sharply with other distributions. Red Hat Enterprise Linux (and Fedora) offer a mature, enterprise-grade environment with strong automation support through Ansible roles for SELinux configuration [8]. Debian provides a conservative, easy-to-use experience with a focus on broad compatibility [7]. However, for the system administrator who values ultimate control, performance tuning, and the ability to build a system from the ground up to meet highly specific security requirements, Gentoo Linux offers an unparalleled environment for SELinux. The journey is more demanding, requiring a deeper understanding of the kernel, the policy language, and the audit process. But the reward is a system that is not just secure, but perfectly attuned to its intended purpose, embodying the true spirit of a customizable and powerful Linux distribution [41].

Feature	Gentoo	Red Hat Enterprise Linux (RHEL/ Fedora)	Debian
Policy Management	Manual via `semanage`, `setsebool`, `audit2allow`. Heavy reliance on USE flags for package integration [42].	Strong focus on automation via Ansible roles (e.g., `rhel-system-roles.selinux`) for large-scale deployments [8].	Manual management with `semanage` and `setsebool`. Less emphasis on Ansible roles compared to RHEL [7].
Package Integration	Granular control via Portage USE flags. SELinux features are conditionally built into packages [42].	SELinux support is often enabled by default in packages. Some packages may require specific flags [19].	Similar to RHEL, with some packages requiring specific flags [7].
User-Friendliness	High learning curve, but offers ultimate control and customization [41].	Good balance, with extensive documentation and community support. More accessible for beginners [8].	Focused on ease of use, but SELinux is less prominent than in RHEL [7].
Philosophy	User-centric control and performance tuning. Build a system from the ground up to meet specific requirements.	Stability, scalability, and automated configuration management. Provide a mature, well-supported platform out-of-the-box.	Broad compatibility and ease of use, with a focus on a stable and predictable system.

Reference

1. 5.3. Understanding Domain Transitions: sepolicy transition https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/security-enhanced_linux-the-sepolicy-suite-sepolicy_transition
2. linux - SELinux - Domain transition failing - Super User https://superuser.com/questions/1851450/selinux-domain-transition-failing
3. Avoiding Executable Duplication for Per-Process Contexts in SELinux https://serverfault.com/questions/1189517/avoiding-executable-duplication-for-per-process-contexts-in-selinux
4. Chapter 2. SELinux Contexts | Red Hat Enterprise Linux | 7 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/chap-security-enhanced_linux-selinux_contexts
5. Selinux Denies Starting Service on Android 8 - Stack Overflow https://stackoverflow.com/questions/48272871/selinux-denies-starting-service-on-android-8
6. SELinux - Arch Linux 中文维基 https://wiki.archlinuxcn.org/wiki/SELinux
7. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=gentoo&secondlist=ubuntudp&firstversions=0&secondversions=0&showall=yes
8. SELinux-系统管理指南-全- - 绝不原创的飞龙- 博客园 https://www.cnblogs.com/apachecn/p/18965749
9. Linux-管理秘籍-全- - 绝不原创的飞龙- 博客园 https://www.cnblogs.com/apachecn/p/18196635
10. 精通-Linux-管理第二版-全 - 博客园 https://www.cnblogs.com/apachecn/p/18967733
11. 如何利用audit2allow 工具调试SELinux日志 - CSDN博客 https://blog.csdn.net/2401_84168288/article/details/148511032
12. Linux Compressed | PDF - Scribd https://www.scribd.com/document/748095195/Linux-Compressed
13. Simple Index - SUSTech Open Source Mirrors https://mirrors.sustech.edu.cn/pypi/simple/

14. Linux Sea | PDF | Operating System - Scribd https://www.scribd.com/document/202134778/linux-sea
15. Manage the kernel on a Linode - Akamai TechDocs https://techdocs.akamai.com/cloud-computing/docs/manage-the-kernel-on-a-compute-instance
16. mkosi.news(7) - Debian Manpages https://manpages.debian.org/testing/mkosi/mkosi.news.7.en.html
17. [XML] https://yum.oracle.com/repo/OracleLinux/OL9/0/baseos/base ... https://yum.oracle.com/repo/OracleLinux/OL9/0/baseos/base/aarch64/repodata/f8f8141c7dd99778773c7bce72dbc7675cd441f4-updateinfo.xml.gz
18. Unable to emerge SystemD on Hardened/SELinux https://unix.stackexchange.com/questions/249021/unable-to-emerge-systemd-on-hardened-selinux
19. 13.3. Booleans | SELinux User's and Administrator's Guide https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/sect-managing_confined_services-the_apache_http_server-booleans
20. How much does httpd_can_network_connect being set to 1 ... https://serverfault.com/questions/1020429/how-much-does-httpd-can-network-connect-being-set-to-1-actually-open-up-on-selin
21. php - detecting if someone has selinux installed ... https://stackoverflow.com/questions/30271436/detecting-if-someone-has-selinux-installed-httpd-can-network-connect-enabled
22. selinux audit log flooded with avc denials to /tmp tmpfs when user is ... https://stackoverflow.com/questions/76472714/selinux-audit-log-flooded-with-avc-denials-to-tmp-tmpfs-when-user-is-set-to-use
23. audit2allow安装与使用（解决avc:denied问题） - 君雁- 博客园 https://www.cnblogs.com/Jadore/p/17349538.html
24. 深入解析audit2allow：从日志分析到SELinux权限修复实战 - CSDN博客 https://blog.csdn.net/weixin_29206669/article/details/157834415
25. [PDF] Release 1.1.9 The Borg Collective https://borgbackup.readthedocs.io/_/downloads/en/1.1.9/pdf/
26. [XML] https://unix.stackexchange.com/sitemap-questions-1.xml https://unix.stackexchange.com/sitemap-questions-1.xml
27. Chapter 2. Introduction | Security-Enhanced Linux | 6 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/security-enhanced_linux/chap-security-enhanced_linux-introduction
28. 4.2. SELinux and Mandatory Access Control (MAC) https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_security_guide/sect-virtualization_security_guide-svirt-mac

29. Chapter 49. Security and SELinux | Red Hat Enterprise Linux | 5 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/5/html/deployment_guide/selg-overview
30. Chapter 21. SELinux | Reference Guide | Red Hat Enterprise Linux | 4 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/4/html/reference_guide/ch-selinux
31. Chapter 1. Introduction | SELinux User's and Administrator's Guide https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/chap-security-enhanced_linux-introduction
32. Chapter 1. Getting started with SELinux | Red Hat Enterprise Linux | 8 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/using_selinux/getting-started-with-selinux_using-selinux
33. Chapter 19. Using SELinux | Red Hat Enterprise Linux | 8 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/system_design_guide/using_selinux
34. SELinux User's and Administrator's Guide | Red Hat Enterprise Linux https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html-single/selinux_users_and_administrators_guide/index
35. 4.13. Multi-Level Security (MLS) | Red Hat Enterprise Linux | 7 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/mls
36. Chapter 3. SELinux Contexts | Security-Enhanced Linux https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/security-enhanced_linux/chap-security-enhanced_linux-selinux_contexts
37. Mastering SELinux: Complete Guide to Security Configuration https://www.coursehero.com/file/250987862/red-hat-enterprise-linux-7-selinux-users-and-administrators-guide-en-uspdf/
38. Fedora-24-SELinux Users and Administrators Guide-en-US - Scribd https://www.scribd.com/document/622538480/Fedora-24-SELinux-Users-and-Administrators-Guide-en-US
39. 13.2. Types | SELinux User's and Administrator's Guide https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/sect-managing_confined_services-the_apache_http_server-types
40. SELinux - Transition Decisions - Oreilly https://www.oreilly.com/library/view/selinux/0596007167/ch02s05.html
41. Is it possible to use systemd on Gentoo with hardened/selinux profile? https://unix.stackexchange.com/questions/588193/is-it-possible-to-use-systemd-on-gentoo-with-hardened-selinux-profile

42. Gentoo Hardening Part 1: Introduction to Hardened Profile - Infosec https://www.infosecinstitute.com/resources/digital-forensics/gentoo-hardening-part-1-introduction-hardened-profile-2/
43. [Android开发技巧] 通过avc日志自动生成selinux策略 - CSDN博客 https://blog.csdn.net/a572423926/article/details/137871542
44. Selinux Audit2allow Guide | PDF | Sudo | Computing - Scribd https://www.scribd.com/document/887014602/selinux-audit2allow-guide
45. Rocky Linux Admin Guide 9.6 | PDF | Gnu | Unix - Scribd https://www.scribd.com/document/789008044/Rocky-Linux-Admin-Guide
46. Distribution Release: Rocky Linux 9.0 (DistroWatch.com News) https://distrowatch.com/11594
47. 鸟哥的Linux私房菜(服务器)- 第七章、网络安全与主机基本防护 https://blog.csdn.net/GarfieldEr007/article/details/50736798
48. 2.4. SELinux States and Modes | Red Hat Enterprise Linux | 6 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/security-enhanced_linux/sect-security-enhanced_linux-introduction-selinux_modes
49. 50.2.6. Enabling or Disabling Enforcement | Red Hat Enterprise Linux https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/5/html/deployment_guide/sec-sel-enable-disable-enforcement
50. SELinux and RHEL: A technical exploration of security hardening https://www.redhat.com/en/blog/selinux-and-rhel-technical-exploration-security-hardening
51. How SELinux improves Red Hat Enterprise Linux security https://developers.redhat.com/articles/2023/09/21/how-selinux-improves-red-hat-enterprise-linux-security
52. Understanding SELinux Basics - SUSE Documentation https://documentation.suse.com/sles-sap/16.0/html/SAP-SELinux/index.html
53. Understanding SELinux Basics | SUSE Linux Enterprise Micro 6.0 https://documentation.suse.com/sle-micro/6.0/html/Micro-selinux/index.html
54. [PDF] Understanding SELinux Basics - SUSE Documentation https://documentation.suse.com/sle-micro/6.2/pdf/Micro-selinux_en.pdf
55. SELinux | SUSE Linux Enterprise Micro Security Guide https://documentation.suse.com/smart/security/html/SLE-Micro-security/ch01.html
56. [PDF] Security Guide - SUSE Linux Enterprise Micro 5.1 https://documentation.suse.com/sle-micro/5.1/pdf/article-selinux_en.pdf
57. Security and Hardening Guide | SLE Micro 5.3 - SUSE Documentation https://documentation.suse.com/sle-micro/5.3/single-html/SLE-Micro-security/

58. ChangeLog-SLE-15-SP4-GM-SLE-15-SP5-PublicRC ... - SUSE https://www.suse.com/c/wp-content/uploads/2023/04/ChangeLog-SLE-15-SP4-GM-SLE-15-SP5-PublicRC-202304.txt
59. Linux 管理秘籍（六）_vendor preset-CSDN博客 https://blog.csdn.net/wizardforcel/article/details/140489795
60. Azure-上的-Linux-管理实用指南-全- - 绝不原创的飞龙- 博客园 https://www.cnblogs.com/apachecn/p/18196551
61. Using SELinux | Red Hat Enterprise Linux | 9 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html-single/using_selinux/index
62. Chapter 3. Targeted Policy | SELinux User's and Administrator's Guide https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/chap-security-enhanced_linux-targeted_policy
63. 50.2.2. Relabeling a File System | Red Hat Enterprise Linux | 5 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/5/html/deployment_guide/sec-sel-fsrelabel
64. SELinux relabeling on boot is stuck - Server Fault https://serverfault.com/questions/948959/selinux-relabeling-on-boot-is-stuck
65. Using SELinux | Red Hat Enterprise Linux | 8 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html-single/using_selinux/index
66. SELinux | Security and Hardening Guide | SLE Micro 5.3 https://documentation.suse.com/sle-micro/5.3/html/SLE-Micro-all/cha-selinux-slemicro.html
67. 统信UOS适配问题经验库 https://ecology.chinauos.com/adaptidentification/knowledge/index.html?status=con2&type1=%E7%B3%BB%E7%BB%9F%E5%B8%B8%E8%A7%81%E9%97%AE%E9%A2%98&type2=%E5%85%B6%E4%BB%96%E7%B3%BB%E7%BB%9F%E9%97%AE%E9%A2%98&num=007582
68. SELinux Notebook | PDF | Computer Access Control - Scribd https://www.scribd.com/document/749878934/SELinux-Notebook
69. 5.6.2. Persistent Changes: semanage fcontext https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/security-enhanced_linux/sect-security-enhanced_linux-selinux_contexts_labeling_files-persistent_changes_semanage_fcontext
70. Semanage fcontext httpd_user_content add new context https://stackoverflow.com/questions/76153329/semanage-fcontext-httpd-user-content-add-new-context
71. restorecon not setting fcontext created using semanage https://unix.stackexchange.com/questions/562536/restorecon-not-setting-fcontext-created-using-semanage

72. Security Guide | SLE Micro 5.1 - SUSE Documentation https://documentation.suse.com/sle-micro/5.1/html/SLE-Micro-all/article-selinux.html
73. 1.4. SELinux States and Modes | Red Hat Enterprise Linux | 7 https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/sect-security-enhanced_linux-introduction-selinux_modes
74. SELinux User's and Administrator's Guide | Red Hat Enterprise Linux https://docs.redhat.com/de/documentation/red_hat_enterprise_linux/7/html-single/selinux_users_and_administrators_guide/index
75. 2365799 – SELinux is preventing systemd-user-ru from connectto ... https://bugzilla.redhat.com/show_bug.cgi?id=2365799
76. How can I do an SELINUX filesystem relabel without rebooting first? https://serverfault.com/questions/453137/how-can-i-do-an-selinux-filesystem-relabel-without-rebooting-first
77. SELinux demands constant relabeling - Unix & Linux Stack Exchange https://unix.stackexchange.com/questions/483246/selinux-demands-constant-relabeling
78. Can't make SELinux context types permanent with semanage https://serverfault.com/questions/551695/cant-make-selinux-context-types-permanent-with-semanage
79. SELinux User's and Administrator's Guide | Red Hat Enterprise Linux https://docs.redhat.com/de/documentation/red_hat_enterprise_linux/7/epub/selinux_users_and_administrators_guide/sect-managing_confined_services-squid_caching_proxy-types
80. SELinux User's and Administrator's Guide | Red Hat Enterprise Linux https://docs.redhat.com/fr/documentation/red_hat_enterprise_linux/7/html-single/selinux_users_and_administrators_guide/index
81. [PDF] SELinux User's and Administrator's Guide - Red Hat Documentation https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/pdf/selinux_users_and_administrators_guide/red_hat_enterprise_linux-7-selinux_users_and_administrators_guide-en-us.pd